

Children's Literacy Learning Platform - Project Overview

Executive Summary

A complete, production-ready backend API for a children's literacy learning platform similar to Duolingo ABC. Built with FastAPI, PostgreSQL, and following clean architecture principles.

Key Statistics

- **54 Python files** across the entire project
- **10 database models** with full relationships
- **7 API route modules** with 30+ endpoints
- **5 test modules** with unit and integration tests
- **Clean layered architecture**: Models → Repositories → Services → Routes

Core Features Implemented

1. Authentication & Authorization

- JWT-based authentication (access + refresh tokens)
- bcrypt password hashing
- Role-based access control (Parent, Admin)
- Token validation middleware
- Secure user registration and login

Files:

- `app/core/security.py` - JWT & password utilities
- `app/services/auth\_service.py` - Authentication logic
- `app/api/routes/auth.py` - Auth endpoints

2. Child Profile Management

- Parents can create multiple child profiles
- Track: name, age, avatar, level, XP, streak
- Full CRUD operations
- Parent-child ownership validation
- Age validation (2-12 years)

Files:

- `app/models/child.py` - Child database model
- `app/services/child\_service.py` - Child business logic
- `app/api/routes/children.py` - Child endpoints

3. Curriculum Management

- Three-tier hierarchy: Units → Lessons → Exercises
- Admin-only CRUD for curriculum content
- Published/unpublished state management
- Ordered content (order_index)
- Multiple exercise types (multiple_choice, fill_blank, etc.)

- JSON storage for flexible exercise content

****Files:****

- `app/models/curriculum.py` - Unit, Lesson, Exercise models
- `app/repositories/curriculum\_repository.py` - Data access
- `app/api/routes/curriculum.py` - Curriculum endpoints

4. Learning Progress Tracking

- Submit exercise answers with validation
- Automatic correctness checking
- Complete lessons with score calculation
- Track all attempts and completions
- Enforce lesson completion rules

****Files:****

- `app/models/learning.py` - ExerciseResult, LessonCompletion
- `app/services/learning\_service.py` - Learning logic
- `app/api/routes/learning.py` - Learning endpoints

5. Gamification System

- ****XP System****: Earn XP for correct exercises and lesson completion
- ****Level Progression****: Auto-level up when XP thresholds met
- ****Streak Tracking****: Daily activity streak with bonus XP
- ****Badge System****: Award badges based on criteria (XP, streak, level)
- ****Automatic Badge Awards****: Triggered after exercises/lessons

****Files:****

- `app/models/gamification.py` - Badge, ChildBadge models
- `app/services/gamification\_service.py` - Gamification logic
- `app/api/routes/gamification.py` - Badge endpoints

6. Real-time Notifications

- WebSocket-based real-time notifications
- Persistent notification storage
- Read/unread status tracking
- Notifications for: badge earned, level up, streak milestones
- Connection management per child

****Files:****

- `app/models/notification.py` - Notification model
- `app/websocket/manager.py` - WebSocket connection manager
- `app/websocket/routes.py` - WebSocket endpoints
- `app/api/routes/notifications.py` - REST notification endpoints

7. Database & Migrations

- PostgreSQL with SQLAlchemy ORM
- Alembic for schema versioning
- Complete initial migration included

- Proper foreign keys and cascading deletes
- Indexed columns for performance

****Files:****

- `alembic/versions/001\_initial\_migration.py` - Initial schema
- `alembic/env.py` - Migration environment
- `app/db/database.py` - Database connection

8. Testing Suite

- pytest configuration
- Test fixtures and database mocking
- Unit tests for business logic
- Integration tests for API endpoints
- Tests for auth, children, learning flows

****Files:****

- `app/tests/conftest.py` - Test configuration
- `app/tests/test\_auth.py` - Auth tests
- `app/tests/test\_children.py` - Child profile tests
- `app/tests/test\_learning.py` - Learning service tests

Architecture Details

Layered Architecture

...

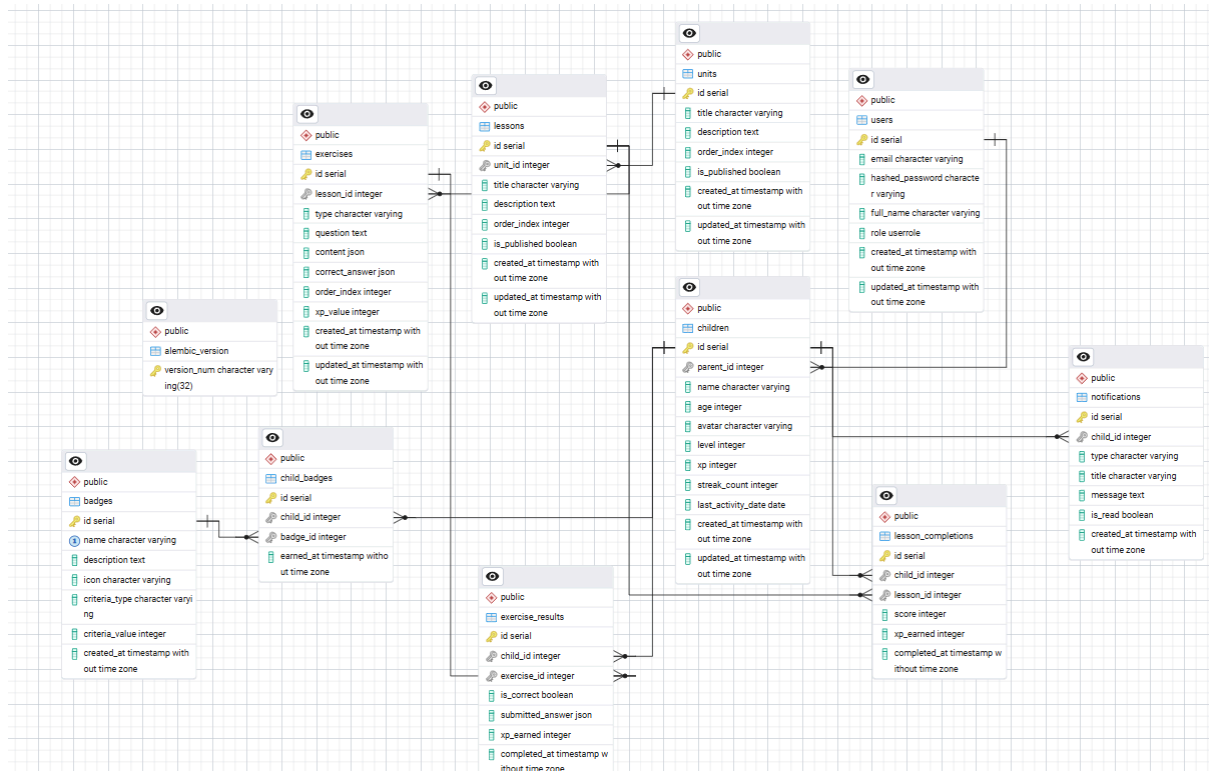
API Routes Layer	← HTTP endpoints, request/response
Services Layer	← Business logic, orchestration
Repositories Layer	← Data access, queries
Models Layer	← SQLAlchemy ORM models
Database (PostgreSQL)	← Data persistence

...

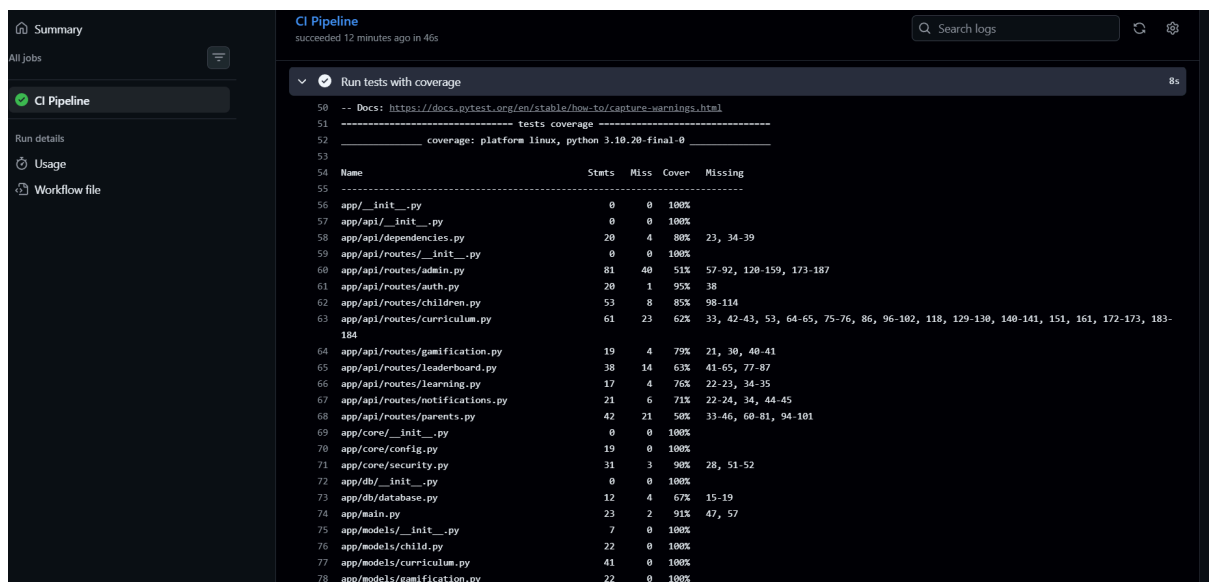
****Benefits:****

- Clear separation of concerns
- Easy to test (mock at each layer)
- Business logic isolated from routes
- Database queries isolated from business logic
- Follows SOLID principles

Entity-Relationship Diagram (ERD)



Testing approach and coverage report screenshot



Data Flow Example: Submit Exercise

...

1. Client → POST /api/v1/learning/children/{id}/exercises
2. Route → Validates JWT, checks ownership

3. Route → Calls LearningService.submit_exercise()
 4. Service → Gets exercise via ExerciseRepository
 5. Service → Checks answer correctness
 6. Service → Creates result via ExerciseResultRepository
 7. Service → Updates child XP via ChildRepository
 8. Service → Calls GamificationService.check_and_award_badges()
 9. Gamification → Checks criteria, awards badges
 10. Gamification → Creates notifications
 11. WebSocket → Sends real-time notification
 12. Service → Returns result
 13. Route → Returns JSON response to client
- ``

Database Schema

Core Tables

****users****

- id, email, hashed_password, full_name, role
- Relationship: One user → Many children

****children****

- id, parent_id, name, age, avatar, level, xp, streak_count
- Relationship: One child → Many results, completions, badges

****units → lessons → exercises****

- Hierarchical curriculum structure
- Published/unpublished state
- Order indexing

****exercise_results****

- Tracks every exercise attempt
- Links: child_id, exercise_id
- Stores: is_correct, submitted_answer, xp_earned

****lesson_completions****

- Tracks completed lessons
- Links: child_id, lesson_id
- Stores: score, xp_earned

****badges → child_badges****

- Badge definitions with criteria
- Many-to-many through child_badges
- Tracks earned_at timestamp

****notifications****

- Per-child notifications
- Type: badge_earned, level_up, etc.

- Read/unread status

API Endpoints Summary

Authentication (3 endpoints)

- POST `/api/v1/auth/register` - Register parent
- POST `/api/v1/auth/login` - Login
- GET `/api/v1/auth/me` - Get current user

Children (5 endpoints)

- POST `/api/v1/children` - Create child
- GET `/api/v1/children` - List children
- GET `/api/v1/children/{id}` - Get child
- PUT `/api/v1/children/{id}` - Update child
- DELETE `/api/v1/children/{id}` - Delete child

Curriculum (12 endpoints)

- Units: CREATE, READ, UPDATE, DELETE
- Lessons: CREATE, READ (by unit), UPDATE, DELETE
- Exercises: CREATE, READ (by lesson), UPDATE, DELETE

Learning (2 endpoints)

- POST `/api/v1/learning/children/{id}/exercises` - Submit exercise
- POST `/api/v1/learning/children/{id}/lessons/{id}/complete` - Complete lesson

Gamification (3 endpoints)

- POST `/api/v1/gamification/badges` - Create badge (admin)
- GET `/api/v1/gamification/badges` - List badges
- GET `/api/v1/gamification/children/{id}/badges` - Child's badges

Notifications (3 endpoints)

- GET `/api/v1/notifications/children/{id}` - Get notifications
- PUT `/api/v1/notifications/{id}/read` - Mark as read
- PUT `/api/v1/notifications/children/{id}/read-all` - Mark all read

WebSocket (1 endpoint)

- WS `/ws/notifications/{child\_id}?token={jwt}` - Real-time notifications

Security Features

1. **Password Security**

- bcrypt hashing (passlib)
- Minimum password requirements enforced by client

2. **JWT Authentication**

- Access tokens (30 min expiry)
- Refresh tokens (7 day expiry)
- HS256 algorithm

- Token type validation

3. ****Authorization****

- Role-based access (Parent, Admin)
- Parent-child ownership checks
- Admin-only curriculum management

4. ****Input Validation****

- Pydantic schemas for all inputs
- Age validation (2-12 years)
- Email validation
- JSON schema validation for exercise content

5. ****Database Security****

- Cascade deletes (orphan cleanup)
- Foreign key constraints
- No SQL injection (ORM parameterization)

Testing Strategy

Unit Tests

- Test services in isolation
- Mock repository layer
- Test business logic (XP, streaks, badges)

Integration Tests

- Test full API flow
- Use test database (SQLite)
- Test authentication flow
- Test ownership validation
- Test error cases

Coverage Areas

- Authentication & authorization
- Child profile CRUD
- Exercise submission & validation
- Lesson completion logic
- Badge award criteria
- Notification creation

Deployment Checklist

- [] Update SECRET_KEY (use: `openssl rand -hex 32`)
- [] Set DEBUG=False
- [] Configure production DATABASE_URL
- [] Set CORS_ORIGINS for frontend domain
- [] Run migrations: `alembic upgrade head`
- [] Seed initial data: `python scripts/seed\_data.py`

- [] Use production WSGI server (gunicorn)
- [] Set up reverse proxy (nginx)
- [] Enable HTTPS/SSL
- [] Configure database backups
- [] Set up monitoring/logging
- [] Configure rate limiting (optional)

Development Workflow

1. ****Adding a new feature****

- Create model in `app/models/`
- Create schema in `app/schemas/`
- Create repository in `app/repositories/`
- Implement service in `app/services/`
- Add routes in `app/api/routes/`
- Write tests in `app/tests/`
- Generate migration: `alembic revision --autogenerate -m "...`"

2. ****Running locally****

```
``bash
source venv/bin/activate
uvicorn app.main:app --reload
``
```

3. ****Running tests****

```
``bash
pytest
pytest --cov=app
``
```

File Count by Category

- ****Models****: 6 files (User, Child, Curriculum, Learning, Gamification, Notification)
- ****Schemas****: 6 files (matching models)
- ****Repositories****: 6 files (data access for each model)
- ****Services****: 4 files (Auth, Child, Learning, Gamification)
- ****Routes****: 7 files (Auth, Children, Curriculum, Learning, Gamification, Notifications, WebSocket)
- ****Tests****: 5 files (conftest, auth, children, learning, + pytest.ini)
- ****Core****: 3 files (config, security, database)
- ****Other****: Scripts, migrations, docs

Next Steps & Extensions

Potential Enhancements

1. ****Admin Dashboard****: Analytics, user management
2. ****Parent Portal****: Track all children, view progress reports
3. ****Content Recommendations****: Suggest lessons based on performance

4. **Leaderboards**: Compare progress (anonymized)
5. **Redis Caching**: Cache curriculum data, reduce DB load
6. **Celery Tasks**: Async email notifications, report generation
7. **File Upload**: Support audio/image exercises
8. **Multi-language**: i18n support for content
9. **OAuth**: Social login (Google, Apple)
10. **Analytics**: Track engagement metrics

Production Scaling

- Add Redis for session storage
- Implement database read replicas
- Add CDN for static assets
- Configure horizontal scaling (multiple workers)
- Add API rate limiting
- Implement request logging and monitoring

Conclusion

This is a **complete, production-ready backend** with:

- ☒ Full authentication & authorization
- ☒ Comprehensive API with 30+ endpoints
- ☒ Clean layered architecture
- ☒ Real-time WebSocket notifications
- ☒ Gamification engine
- ☒ Database migrations
- ☒ Test suite
- ☒ Security best practices
- ☒ Detailed documentation
- ☒ Setup scripts and examples

Ready to deploy with PostgreSQL and can be scaled horizontally.